Catfish Farm

Bu Dengklek owns a catfish farm. The catfish farm is a pond consisting of a $N \times N$ grid of cells. Each cell is a square of the same size. The columns of the grid are numbered from 0 to $N-1$ from west to east and the rows are numbered from 0 to $N-1$ from south to north. We refer to the cell located at column c and row r of the grid ($0 \leq c \leq N-1$, $0 \leq r \leq N-1$) as cell (c, r).

In the pond, there are M catfish, numbered from 0 to $M-1$, located at **distinct** cells. For each i such that $0 \leq i \leq M-1$, catfish i is located at cell $(X[i], Y[i])$, and weighs $W[i]$ grams.

Bu Dengklek wants to build some piers to catch the catfish. A pier in column c of length k (for any $0 \leq c \leq N-1$ and $1 \leq k \leq N$) is a rectangle extending from row 0 to row $k-1$, covering cells $(c, 0), (c, 1), \ldots, (c, k-1)$. For each column, Bu Dengklek can choose either to build a pier of some length of her choice or to not build a pier.

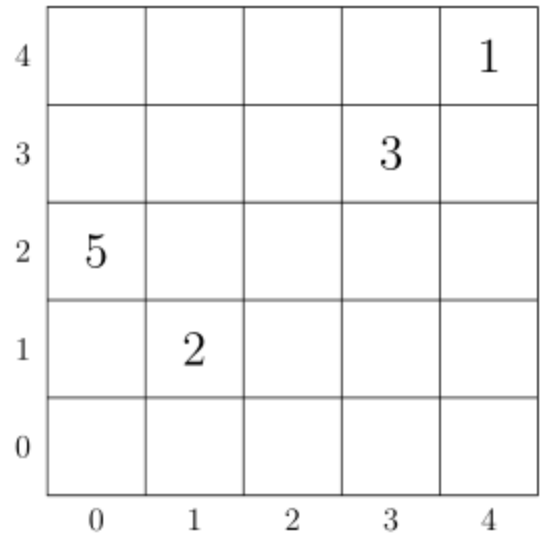
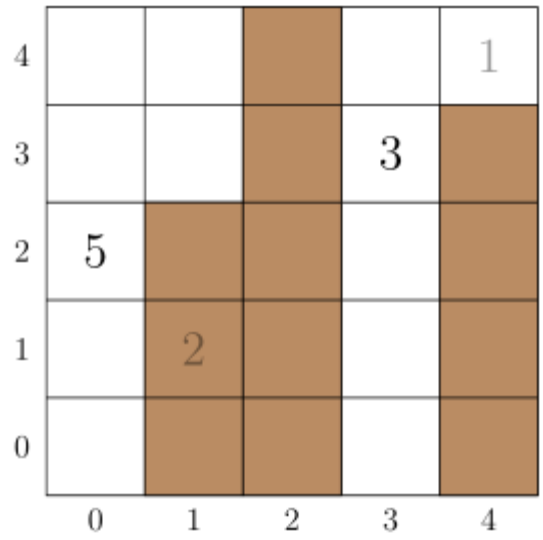
Catfish i (for each i such that $0 \leq i \leq M-1$) can be caught if there is a pier directly to the west or east of it, and there is no pier covering its cell; that is, if

- **at least one** of the cells $(X[i]-1, Y[i])$ or $(X[i]+1, Y[i])$ is covered by a pier, and
- there is no pier covering cell $(X[i], Y[i])$.

For example, consider a pond of size $N = 5$ with $M = 4$ catfish:

- Catfish 0 is located at cell $(0, 2)$ and weighs 5 grams.
- Catfish 1 is located at cell $(1, 1)$ and weighs 2 grams.
- Catfish 2 is located at cell $(4, 4)$ and weighs 1 gram.
- Catfish 3 is located at cell $(3, 3)$ and weighs 3 grams.

One way Bu Dengklek can build the piers is as follows:

Before the piers are built	After the piers are built
	

The number at a cell denotes the weight of the catfish located at the cell. The shaded cells are covered by piers. In this case, catfish 0 (at cell $(0, 2)$) and catfish 3 (at cell $(3, 3)$) can be caught. Catfish 1 (at cell $(1, 1)$) cannot be caught, as there is a pier covering its location, while catfish 2 (at cell $(4, 4)$) can not be caught as there is no pier directly to the west nor east of it.

Bu Dengklek would like to build piers so that the total weight of catfish she can catch is as large as possible. Your task is to find the maximum total weight of catfish that Bu Dengklek can catch after building piers.

Implementation Details

You should implement the following procedure:

```
int64 max_weights(int N, int M, int[] X, int[] Y, int[] W)
```

- N: the size of the pond.
- M: the number of catfish.
- X, Y: arrays of length M describing catfish locations.
- W: array of length M describing catfish weights.
- This procedure should return an integer representing the maximum total weight of catfish that Bu Dengklek can catch after building piers.
- This procedure is called exactly once.

Example

Consider the following call:

```
max_weights(5, 4, [0, 1, 4, 3], [2, 1, 4, 3], [5, 2, 1, 3])
```

This example is illustrated in the task description above.

After building piers as described, Bu Dengklek can catch catfish 0 and 3, whose total weight is $5 + 3 = 8$ grams. As there is no way to build piers to catch catfish with a total weight of more than 8 grams, the procedure should return 8.

Constraints

- $2 \leq N \leq 100\,000$
- $1 \leq M \leq 300\,000$
- $0 \leq X[i] \leq N - 1$, $0 \leq Y[i] \leq N - 1$ (for each i such that $0 \leq i \leq M - 1$)
- $1 \leq W[i] \leq 10^9$ (for each i such that $0 \leq i \leq M - 1$)
- No two catfish share the same cell. In other words, $X[i] \neq X[j]$ or $Y[i] \neq Y[j]$ (for each i and j such that $0 \leq i < j \leq M - 1$).

Subtasks

1. (3 points) $X[i]$ is even (for each i such that $0 \leq i \leq M - 1$)
2. (6 points) $X[i] \leq 1$ (for each i such that $0 \leq i \leq M - 1$)
3. (9 points) $Y[i] = 0$ (for each i such that $0 \leq i \leq M - 1$)
4. (14 points) $N \leq 300$, $Y[i] \leq 8$ (for each i such that $0 \leq i \leq M - 1$)
5. (21 points) $N \leq 300$
6. (17 points) $N \leq 3000$
7. (14 points) There are at most 2 catfish in each column.
8. (16 points) No additional constraints.

Sample Grader

The sample grader reads the input in the following format:

- line 1: N M
- line $2 + i$ ($0 \leq i \leq M - 1$): $X[i]$ $Y[i]$ $W[i]$

The sample grader prints your answer in the following format:

- line 1: the return value of `max_weights`

Longest Trip

The IOI 2023 organizers are in big trouble! They forgot to plan the trip to Ópusztaszer for the upcoming day. But maybe it is not yet too late ...

There are N landmarks at Ópusztaszer indexed from 0 to $N-1$. Some pairs of these landmarks are connected by *bidirectional* **roads**. Each pair of landmarks is connected by at most one road. The organizers *don't know* which landmarks are connected by roads.

We say that the **density** of the road network at Ópusztaszer is **at least** δ if every 3 distinct landmarks have at least δ roads among them. In other words, for each triplet of landmarks (u, v, w) such that $0 \leq u < v < w < N$, among the pairs of landmarks $(u, v), (v, w)$ and (u, w) at least δ pairs are connected by a road.

The organizers *know* a positive integer D such that the density of the road network is at least D. Note that the value of D cannot be greater than 3.

The organizers can make **calls** to the phone dispatcher at Ópusztaszer to gather information about the road connections between certain landmarks. In each call, two nonempty arrays of landmarks $[A[0], \ldots, A[P-1]]$ and $[B[0], \ldots, B[R-1]]$ must be specified. The landmarks must be pairwise distinct, that is,

- $A[i] \neq A[j]$ for each i and j such that $0 \leq i < j < P$;
- $B[i] \neq B[j]$ for each i and j such that $0 \leq i < j < R$;
- $A[i] \neq B[j]$ for each i and j such that $0 \leq i < P$ and $0 \leq j < R$.

For each call, the dispatcher reports whether there is a road connecting a landmark from A and a landmark from B. More precisely, the dispatcher iterates over all pairs i and j such that $0 \leq i < P$ and $0 \leq j < R$. If, for any of them, the landmarks $A[i]$ and $B[j]$ are connected by a road, the dispatcher returns `true`. Otherwise, the dispatcher returns `false`.

A **trip** of length l is a sequence of *distinct* landmarks $t[0], t[1], \ldots, t[l-1]$, where for each i between 0 and $l-2$, inclusive, landmark $t[i]$ and landmark $t[i+1]$ are connected by a road. A trip of length l is called a **longest trip** if there does not exist any trip of length at least $l+1$.

Your task is to help the organizers to find a longest trip at Ópusztaszer by making calls to the dispatcher.

Implementation Details

You should implement the following procedure:

```
int[] longest_trip(int N, int D)
```

- N: the number of landmarks at Ópusztaszer.
- D: the guaranteed minimum density of the road network.
- This procedure should return an array $t = [t[0], t[1], \ldots, t[l-1]]$, representing a longest trip.
- This procedure may be called **multiple times** in each test case.

The above procedure can make calls to the following procedure:

```
bool are_connected(int[] A, int[] B)
```

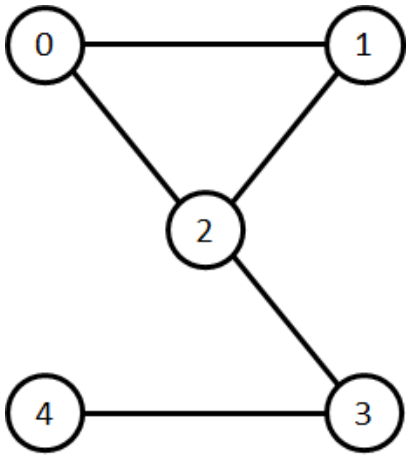
- A: a nonempty array of distinct landmarks.
- B: a nonempty array of distinct landmarks.
- A and B should be disjoint.
- This procedure returns `true` if there is a landmark from A and a landmark from B connected by a road. Otherwise, it returns `false`.
- This procedure can be called at most $32\,640$ times in each invocation of `longest_trip`, and at most $150\,000$ times in total.
- The total length of arrays A and B passed to this procedure over all of its invocations cannot exceed $1\,500\,000$.

The grader is **not adaptive**. Each submission is graded on the same set of test cases. That is, the values of N and D, as well as the pairs of landmarks connected by roads, are fixed for each call of `longest_trip` within each test case.

Examples

Example 1

Consider a scenario in which $N = 5$, $D = 1$, and the road connections are as shown in the following figure:



The procedure `longest_trip` is called in the following way:

```
longest_trip(5, 1)
```

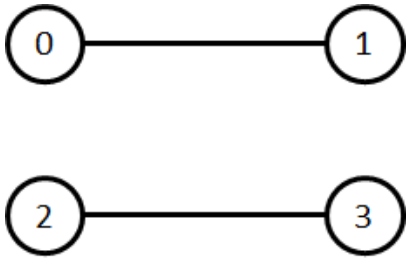
The procedure may make calls to `are_connected` as follows.

Call	Pairs connected by a road	Return value
`are_connected([0], [1, 2, 4, 3])`	$(0, 1)$ and $(0, 2)$	`true`
`are_connected([2], [0])`	$(2, 0)$	`true`
`are_connected([2], [3])`	$(2, 3)$	`true`
`are_connected([1, 0], [4, 3])`	none	`false`

After the fourth call, it turns out that *none* of the pairs $(1, 4)$, $(0, 4)$, $(1, 3)$ and $(0, 3)$ is connected by a road. As the density of the network is at least $D = 1$, we see that from the triplet $(0, 3, 4)$, the pair $(3, 4)$ must be connected by a road. Similarly to this, landmarks 0 and 1 must be connected.

At this point, it can be concluded that $t = [1, 0, 2, 3, 4]$ is a trip of length 5, and that there does not exist a trip of length greater than 5. Therefore, the procedure `longest_trip` may return $[1, 0, 2, 3, 4]$.

Consider another scenario in which $N = 4$, $D = 1$, and the roads between the landmarks are as shown in the following figure:



The procedure `longest_trip` is called in the following way:

```
longest_trip(4, 1)
```

In this scenario, the length of a longest trip is 2. Therefore, after a few calls to procedure `are_connected`, the procedure `longest_trip` may return one of $[0, 1]$, $[1, 0]$, $[2, 3]$ or $[3, 2]$.

Example 2

Subtask 0 contains an additional example test case with $N = 256$ landmarks. This test case is included in the attachment package that you can download from the contest system.

Constraints

- $3 \leq N \leq 256$
- The sum of N over all calls to `longest_trip` does not exceed $1\,024$ in each test case.
- $1 \leq D \leq 3$

Subtasks

1. (5 points) $D = 3$
2. (10 points) $D = 2$
3. (25 points) $D = 1$. Let $l^\star$ denote the length of a longest trip. Procedure `longest_trip` does not have to return a trip of length $l^\star$. Instead, it should return a trip of length at least $\left\lceil \frac{l^\star}{2} \right\rceil$.
4. (60 points) $D = 1$

In subtask 4 your score is determined based on the number of calls to procedure `are_connected` over a single invocation of `longest_trip`. Let q be the maximum number of calls among all invocations of `longest_trip` over every test case of the subtask. Your score for this subtask is calculated according to the following table:

Condition	Points
$2\,750 < q \leq 32\,640$	20
$550 < q \leq 2\,750$	30
$400 < q \leq 550$	45
$q \leq 400$	60

If, in any of the test cases, the calls to the procedure `are_connected` do not conform to the constraints described in Implementation Details, or the array returned by `longest_trip` is incorrect, the score of your solution for that subtask will be 0.

Sample Grader

Let C denote the number of scenarios, that is, the number of calls to `longest_trip`. The sample grader reads the input in the following format:

- line 1: C

The descriptions of C scenarios follow.

The sample grader reads the description of each scenario in the following format:

- line 1: $N\ D$
- line $1+i$ ($1 \le i < N$): $U_i[0]\ U_i[1]\ \ldots\ U_i[i-1]$

Here, each U_i ($1 \le i < N$) is an array of size i, describing which pairs of landmarks are connected by a road. For each i and j such that $1 \le i < N$ and $0 \le j < i$:

- if landmarks j and i are connected by a road, then the value of $U_i[j]$ should be 1;
- if there is no road connecting landmarks j and i, then the value of $U_i[j]$ should be 0.

In each scenario, before calling `longest_trip`, the sample grader checks whether the density of the road network is at least D. If this condition is not met, it prints the message `Insufficient Density` and terminates.

If the sample grader detects a protocol violation, the output of the sample grader is `Protocol Violation: <MSG>`, where `<MSG>` is one of the following error messages:

- `invalid array`: in a call to `are_connected`, at least one of arrays A and B
  - is empty, or
  - contains an element that is not an integer between 0 and $N-1$, inclusive, or
  - contains the same element at least twice.
- `non-disjoint arrays`: in a call to `are_connected`, arrays A and B are not disjoint.
- `too many calls`: the number of calls made to `are_connected` exceeds $32\,640$ over the current invocation of `longest trip`, or exceeds $150\,000$ in total.
- `too many elements`: the total number of landmarks passed to `are_connected` over all calls exceeds $1\,500\,000$.

Otherwise, let the elements of the array returned by `longest_trip` in a scenario be $t[0], t[1], \ldots, t[l-1]$ for some nonnegative l. The sample grader prints three lines for this scenario in the following format:

- line 1: l
- line 2: $t[0]\ t[1]\ \ldots\ t[l-1]$
- line 3: the number of calls to `are_connected` over this scenario

Finally, the sample grader prints:

- line $1 + 3 \cdot C$: the maximum number of calls to `are_connected` over all calls to `longest_trip`

Souvenirs

Amaru is buying souvenirs in a foreign shop. There are N **types** of souvenirs. There are infinitely many souvenirs of each type available in the shop.

Each type of souvenir has a fixed price. Namely, a souvenir of type i ($0 \leq i < N$) has a price of $P[i]$ coins, where $P[i]$ is a positive integer.

Amaru knows that souvenir types are sorted in decreasing order by price, and that the souvenir prices are distinct. Specifically, $P[0] > P[1] > \cdots > P[N-1] > 0$. Moreover, he was able to learn the value of $P[0]$. Unfortunately, Amaru does not have any other information about the prices of the souvenirs.

To buy some souvenirs, Amaru will perform a number of transactions with the seller.

Each transaction consists of the following steps:

1. Amaru hands some (positive) number of coins to the seller.
2. The seller puts these coins in a pile on the table in the back room, where Amaru cannot see them.
3. The seller considers each souvenir type $0, 1, \ldots, N-1$ in that order, one by one. Each type is considered **exactly once** per transaction.
    - When considering souvenir type i, if the current number of coins in the pile is at least $P[i]$, then
        - the seller removes $P[i]$ coins from the pile, and
        - the seller puts one souvenir of type i on the table.
4. The seller gives Amaru all the coins remaining in the pile and all souvenirs on the table.

Note that there are no coins or souvenirs on the table before a transaction begins.

Your task is to instruct Amaru to perform some number of transactions, so that:

- in each transaction he buys **at least one** souvenir, and
- overall he buys **exactly** i souvenirs of type i, for each i such that $0 \leq i < N$. Note that this means that Amaru should not buy any souvenir of type 0.

Amaru does not have to minimize the number of transactions and has an unlimited supply of coins.

Implementation Details

You should implement the following procedure.

```
void buy_souvenirs(int N, long long P0)
```

- N: the number of souvenir types.
- $P0$: the value of $P[0]$.
- This procedure is called exactly once for each test case.

The above procedure can make calls to the following procedure to instruct Amaru to perform a transaction:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M: the number of coins handed to the seller by Amaru.
- The procedure returns a pair. The first element of the pair is an array L, containing the types of souvenirs that have been bought (in increasing order). The second element is an integer R, the number of coins returned to Amaru after the transaction.
- It is required that $P[0] > M \geq P[N-1]$. The condition $P[0] > M$ ensures that Amaru does not buy any souvenir of type 0, and $M \geq P[N-1]$ ensures that Amaru buys at least one souvenir. If these conditions are not met, your solution will receive the verdict `Output isn't correct: Invalid argument`. Note that contrary to $P[0]$, the value of $P[N-1]$ is not provided in the input.
- The procedure can be called at most 5000 times in each test case.

The behavior of the grader is **not adaptive**. This means that the sequence of prices P is fixed before `buy_souvenirs` is called.

Constraints

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ for each i such that $0 \leq i < N$.
- $P[i] > P[i+1]$ for each i such that $0 \leq i < N-1$.

Subtasks

Subtask	Score	Additional Constraints
1	4	$N = 2$
2	3	$P[i] = N - i$ for each i such that $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ for each i such that $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ for each i such that $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ for each i such that $0 \leq i < N - 1$.
6	33	No additional constraints.

Example

Consider the following call.

```
buy_souvenirs(3, 4)
```

There are $N = 3$ types of souvenirs and $P[0] = 4$. Observe that there are only three possible sequences of prices P: $[4, 3, 2]$, $[4, 3, 1]$, and $[4, 2, 1]$.

Assume that `buy_souvenirs` calls `transaction(2)`. Suppose the call returns $([2], 1)$, meaning that Amaru bought one souvenir of type 2 and the seller gave him back 1 coin. Observe that this allows us to deduce that $P = [4, 3, 1]$, since:

- For $P = [4, 3, 2]$, `transaction(2)` would have returned $([2], 0)$.
- For $P = [4, 2, 1]$, `transaction(2)` would have returned $([1], 0)$.

Then `buy_souvenirs` can call `transaction(3)`, which returns $([1], 0)$, meaning that Amaru bought one souvenir of type 1 and the seller gave him back 0 coins. So far, in total, he has bought one souvenir of type 1 and one souvenir of type 2.

Finally, `buy_souvenirs` can call `transaction(1)`, which returns $([2], 0)$, meaning that Amaru bought one souvenir of type 2. Note that we could have also used `transaction(2)` here. At this point, in total Amaru has one souvenir of type 1 and two souvenirs of type 2, as required.

Sample Grader

Input format:

```
N
P[0] P[1] ... P[N-1]
```

Output format:

```
Q[0] Q[1] ... Q[N-1]
```

Here $Q[i]$ is the number of souvenirs of type i bought in total for each i such that $0 \leq i < N$.

Digital Circuit

There is a circuit, which consists of $N + M$ **gates** numbered from 0 to $N + M - 1$. Gates 0 to $N - 1$ are **threshold gates**, whereas gates N to $N + M - 1$ are **source gates**.

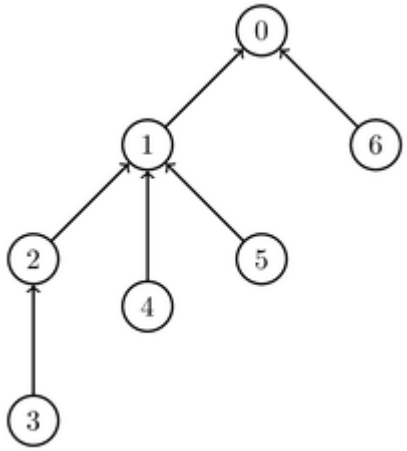
Each gate, except for gate 0, is an **input** to exactly one threshold gate. Specifically, for each i such that $1 \leq i \leq N + M - 1$, gate i is an input to gate $P[i]$, where $0 \leq P[i] \leq N - 1$. Importantly, we also have $P[i] < i$. Moreover, we assume $P[0] = -1$. Each threshold gate has one or more inputs. Source gates do not have any inputs.

Each gate has a **state** which is either 0 or 1. The initial states of the source gates are given by an array A of M integers. That is, for each j such that $0 \leq j \leq M - 1$, the initial state of the source gate $N + j$ is $A[j]$.

The state of each threshold gate depends on the states of its inputs and is determined as follows. First, each threshold gate is assigned a threshold **parameter**. The parameter assigned to a threshold gate with c inputs must be an integer between 1 and c (inclusive). Then, the state of a threshold gate with parameter p is 1, if at least p of its inputs have state 1, and 0 otherwise.

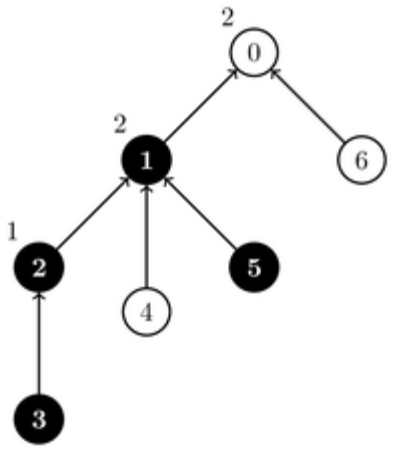
For example, suppose there are $N = 3$ threshold gates and $M = 4$ source gates. The inputs to gate 0 are gates 1 and 6, the inputs to gate 1 are gates 2, 4, and 5, and the only input to gate 2 is gate 3.

This example is illustrated in the following picture.



Suppose that source gates 3 and 5 have state 1, while source gates 4 and 6 have state 0. Assume we assign parameters 1, 2 and 2 to threshold gates 2, 1 and 0 respectively. In this case, gate 2 has

state 1, gate 1 has state 1 and gate 0 has state 0. This assignment of parameter values and the states is illustrated in the following picture. Gates whose state is 1 are marked in black.



The states of the source gates will undergo Q updates. Each update is described by two integers L and R ($N \le L \le R \le N + M - 1$) and toggles the states of all source gates numbered between L and R, inclusive. That is, for each i such that $L \le i \le R$, source gate i changes its state to 1, if its state is 0, or to 0, if its state is 1. The new state of each toggled gate remains unchanged until it is possibly toggled by one of the later updates.

Your goal is to count, after each update, how many different assignments of parameters to threshold gates result in gate 0 having state 1. Two assignments are considered different if there exists at least one threshold gate that has a different value of its parameter in both assignments. As the number of ways can be large, you should compute it modulo $1\,000\,002\,022$.

Note that in the example above, there are 6 different assignments of parameters to threshold gates, since gates 0, 1 and 2 have 2, 3 and 1 inputs respectively. In 2 out of these 6 assignments, gate 0 has state 1.

Implementation Details

Your task is to implement two procedures.

```
void init(int N, int M, int[] P, int[] A)
```

- N: the number of threshold gates.
- M: the number of source gates.
- P: an array of length $N + M$ describing the inputs to the threshold gates.
- A: an array of length M describing the initial states of the source gates.
- This procedure is called exactly once, before any calls to `count_ways`.

```
int count_ways(int L, int R)
```

- L, R: the boundaries of the range of source gates, whose states are toggled.
- This procedure should first perform the specified update, and then return the number of ways, modulo $1\,000\,002\,022$, of assigning parameters to the threshold gates, which result in gate 0 having state 1.
- This procedure is called exactly Q times.

Example

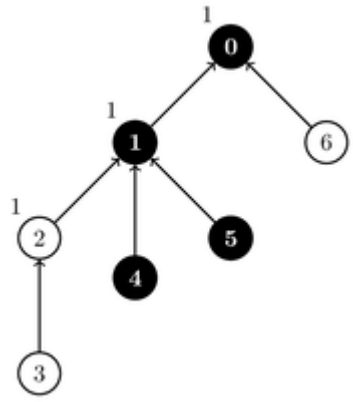
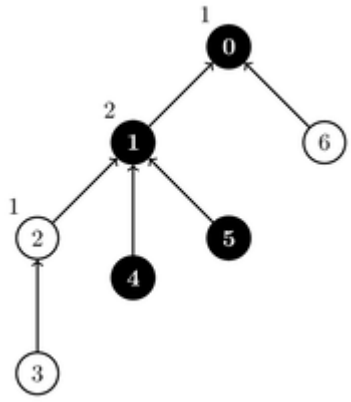
Consider the following sequence of calls:

```
init(3, 4, [-1, 0, 1, 2, 1, 1, 0], [1, 0, 1, 0])
```

This example is illustrated in the task description above.

```
count_ways(3, 4)
```

This toggles the states of gates 3 and 4, i.e. the state of gate 3 becomes 0, and the state of gate 4 becomes 1. Two ways of assigning the parameters which result in gate 0 having state 1 are illustrated in the pictures below.

Way 1	**Way** 2
	

In all other assignments of parameters, gate 0 has state 0. Thus, the procedure should return 2.

```
count_ways(4, 5)
```

This toggles the states of gates 4 and 5. As a result, all source gates have state 0, and for any assignment of parameters, gate 0 has state 0. Thus, the procedure should return 0.

```
count_ways(3, 6)
```

This changes the states of all source gates to 1. As a result, for any assignment of parameters, gate 0 has state 1. Thus, the procedure should return 6.

Constraints

- $1 \leq N, M \leq 100\,000$
- $1 \leq Q \leq 100\,000$
- $P[0] = -1$
- $0 \leq P[i] < i$ and $P[i] \leq N - 1$ (for each i such that $1 \leq i \leq N + M - 1$)
- Each threshold gate has at least one input (for each i such that $0 \leq i \leq N - 1$ there exists an index x such that $i < x \leq N + M - 1$ and $P[x] = i$).
- $0 \leq A[j] \leq 1$ (for each j such that $0 \leq j \leq M - 1$)
- $N \leq L \leq R \leq N + M - 1$

Subtasks

1. (2 points) $N = 1$, $M \leq 1000$, $Q \leq 5$
2. (7 points) $N, M \leq 1000$, $Q \leq 5$, each threshold gate has exactly two inputs.
3. (9 points) $N, M \leq 1000$, $Q \leq 5$
4. (4 points) $M = N + 1$, $M = 2^z$ (for some positive integer z), $P[i] = \lfloor \frac{i-1}{2} \rfloor$ (for each i such that $1 \leq i \leq N + M - 1$), $L = R$
5. (12 points) $M = N + 1$, $M = 2^z$ (for some positive integer z), $P[i] = \lfloor \frac{i-1}{2} \rfloor$ (for each i such that $1 \leq i \leq N + M - 1$)
6. (27 points) Each threshold gate has exactly two inputs.
7. (28 points) $N, M \leq 5000$
8. (11 points) No additional constraints.

Sample Grader

The sample grader reads the input in the following format:

- line 1: $N\ M\ Q$
- line 2: $P[0]\ P[1]\ \ldots\ P[N + M - 1]$
- line 3: $A[0]\ A[1]\ \ldots\ A[M - 1]$
- line $4 + k$ ($0 \leq k \leq Q - 1$): $L\ R$ for update k

The sample grader prints your answers in the following format:

- line $1 + k$ ($0 \leq k \leq Q - 1$): the return value of `count_ways` for update k

Prisoner Challenge

In a prison, there are 500 prisoners. One day, the warden offers them a chance to free themselves. He places two bags with money, bag A and bag B, in a room. Each bag contains between 1 and N coins, inclusive. The number of coins in bag A is **different** from the number of coins in bag B. The warden presents the prisoners with a challenge. The goal of the prisoners is to identify the bag with fewer coins.

The room, in addition to the money bags, also contains a whiteboard. A single number must be written on the whiteboard at any time. Initially, the number on the whiteboard is 0.

Then, the warden asks the prisoners to enter the room, one by one. The prisoner who enters the room does not know which or how many other prisoners have entered the room before them. Every time a prisoner enters the room, they read the number currently written on the whiteboard. After reading the number, they must choose either bag A or bag B. The prisoner then **inspects** the chosen bag, thus getting to know the number of coins inside it. Then, the prisoner must perform either of the following two **actions**:

- Overwrite the number on the whiteboard with a non-negative integer and leave the room. Note that they can either change or keep the current number. The challenge continues after that (unless all 500 prisoners have already entered the room).
- Identify one bag as the one that has fewer coins. This immediately ends the challenge.

The warden will not ask a prisoner who has left the room to enter the room again.

The prisoners win the challenge if one of them correctly identifies the bag with fewer coins. They lose if any of them identifies the bag incorrectly, or all 500 of them have entered the room and not attempted to identify the bag with fewer coins.

Before the challenge starts, the prisoners gather in the prison hall and decide on a common **strategy** for the challenge in three steps.

- They pick a non-negative integer x, which is the largest number they may ever want to write on the whiteboard.
- They decide, for any number i written on the whiteboard ($0 \leq i \leq x$), which bag should be inspected by a prisoner who reads number i from the whiteboard upon entering the room.
- They decide what action a prisoner in the room should perform after getting to know the number of coins in the chosen bag. Specifically, for any number i written on the whiteboard

($0 \leq i \leq x$) and any number of coins j seen in the inspected bag ($1 \leq j \leq N$), they either decide
    - what number between 0 and x (inclusive) should be written on the whiteboard, or
    - which bag should be identified as the one with fewer coins.

Upon winning the challenge, the warden will release the prisoners after serving x more days.

Your task is to devise a strategy for the prisoners that would ensure they win the challenge (regardless of the number of coins in bag A and bag B). The score of your solution depends on the value of x (see Subtasks section for details).

Implementation Details

You should implement the following procedure:

```
int[][] devise_strategy(int N)
```

- N: the maximum possible number of coins in each bag.
- This procedure should return an array s of arrays of $N + 1$ integers, representing your strategy. The value of x is the length of array s minus one. For each i such that $0 \leq i \leq x$, the array $s[i]$ represents what a prisoner should do if they read number i from the whiteboard upon entering the room:
    1. The value of $s[i][0]$ is 0 if the prisoner should inspect bag A, or 1 if the prisoner should inspect bag B.
    2. Let j be the number of coins seen in the chosen bag. The prisoner should then perform the following action:
        - If the value of $s[i][j]$ is -1, the prisoner should identify bag A as the one with fewer coins.
        - If the value of $s[i][j]$ is -2, the prisoner should identify bag B as the one with fewer coins.
        - If the value of $s[i][j]$ is a non-negative number, the prisoner should write that number on the whiteboard. Note that $s[i][j]$ must be at most x.
- This procedure is called exactly once.

Example

Consider the following call:

```
devise_strategy(3)
```

Let v denote the number the prisoner reads from the whiteboard upon entering the room. One of the correct strategies is as follows:

- If $v = 0$ (including the initial number), inspect bag A.

    - If it contains 1 coin, identify bag A as the one with fewer coins.
    - If it contains 3 coins, identify bag B as the one with fewer coins.
    - If it contains 2 coins, write 1 on the whiteboard (overwriting 0).
- If $v = 1$, inspect bag B.
    - If it contains 1 coin, identify bag B as the one with fewer coins.
    - If it contains 3 coins, identify bag A as the one with fewer coins.
    - If it contains 2 coins, write 0 on the whiteboard (overwriting 1). Note that this case can never happen, as we can conclude that both bags contain 2 coins, which is not allowed.

To report this strategy the procedure should return `[[0, -1, 1, -2], [1, -2, 0, -1]]`. The length of the returned array is 2, so for this return value the value of x is $2 - 1 = 1$.

Constraints

- $2 \leq N \leq 5000$

Subtasks

1. (5 points) $N \leq 500$, the value of x must not be more than 500.
2. (5 points) $N \leq 500$, the value of x must not be more than 70.
3. (90 points) The value of x must not be more than 60.

If in any of the test cases, the array returned by `devise_strategy` does not represent a correct strategy, the score of your solution for that subtask will be 0.

In subtask 3 you can obtain a partial score. Let m be the maximum value of x for the returned arrays over all test cases in this subtask. Your score for this subtask is calculated according to the following table:

Condition	Points
$40 \leq m \leq 60$	20
$26 \leq m \leq 39$	$25 + 1.5 \times (40 - m)$
$m = 25$	50
$m = 24$	55
$m = 23$	62
$m = 22$	70
$m = 21$	80
$m \leq 20$	90

Sample Grader

The sample grader reads the input in the following format:

- line 1: N
- line $2 + k$ $(0 \leq k)$: $A[k]$ $B[k]$
- last line: -1

Each line except the first and the last one represents a scenario. We refer to the scenario described in line $2 + k$ as scenario k. In scenario k bag A contains $A[k]$ coins and bag B contains $B[k]$ coins.

The sample grader first calls `devise_strategy(N)`. The value of x is the length of the array returned by the call minus one. Then, if the sample grader detects that the array returned by `devise_strategy` does not conform to the constraints described in Implementation Details, it prints one of the following error messages and exits:

- `s is an empty array`: s is an empty array (which does not represent a valid strategy).
- `s[i] contains incorrect length`: There exists an index i $(0 \leq i \leq x)$ such that the length of $s[i]$ is not $N + 1$.
- `First element of s[i] is non-binary`: There exists an index i $(0 \leq i \leq x)$ such that $s[i][0]$ is neither 0 nor 1.
- `s[i][j] contains incorrect value`: There exist indices i, j $(0 \leq i \leq x, 1 \leq j \leq N)$ such that $s[i][j]$ is not between -2 and x.

Otherwise, the sample grader produces two outputs.

First, the sample grader prints the output of your strategy in the following format:

- line $1 + k$ $(0 \leq k)$: output of your strategy for scenario k. If applying the strategy leads to a prisoner identifying bag A as the one with fewer coins, then the output is the character `A`. If applying the strategy leads to a prisoner identifying bag B as the one with fewer coins, then the output is the character `B`. If applying the strategy does not lead to any prisoner identifying a bag with fewer coins, then the output is the character `X`.

Second, the sample grader writes a file `log.txt` in the current directory in the following format:

- line $1 + k$ $(0 \leq k)$: $w[k][0]$ $w[k][1]$ $\ldots$

The sequence on line $1 + k$ corresponds to scenario k and describes the numbers written on the whiteboard. Specifically, $w[k][l]$ is the number written by the $(l + 1)^{th}$ prisoner to enter the room.

Radio Towers

There are N radio towers in Jakarta. The towers are located along a straight line and numbered from 0 to $N-1$ from left to right. For each i such that $0 \leq i \leq N-1$, the height of tower i is $H[i]$ metres. The heights of the towers are **distinct**.

For some positive interference value δ, a pair of towers i and j (where $0 \leq i < j \leq N-1$) can communicate with each other if and only if there is an intermediary tower k, such that

- tower i is to the left of tower k and tower j is to the right of tower k, that is, $i < k < j$, and
- the heights of tower i and tower j are both at most $H[k] - \delta$ metres.

Pak Dengklek wants to lease some radio towers for his new radio network. Your task is to answer Q questions of Pak Dengklek which are of the following form: given parameters L, R and D ($0 \leq L \leq R \leq N-1$ and $D > 0$), what is the maximum number of towers Pak Dengklek can lease, assuming that

- Pak Dengklek can only lease towers with indices between L and R (inclusive), and
- the interference value δ is D, and
- any pair of radio towers that Pak Dengklek leases must be able to communicate with each other.

Note that two leased towers may communicate using an intermediary tower k, regardless of whether tower k is leased or not.

Implementation Details

You should implement the following procedures:

```
void init(int N, int[] H)
```

- N: the number of radio towers.
- H: an array of length N describing the tower heights.
- This procedure is called exactly once, before any calls to `max_towers`.

```
int max_towers(int L, int R, int D)
```

- L, R: the boundaries of a range of towers.
- D: the value of δ.

- This procedure should return the maximum number of radio towers Pak Dengklek can lease for his new radio network if he is only allowed to lease towers between tower L and tower R (inclusive) and the value of δ is D.
- This procedure is called exactly Q times.

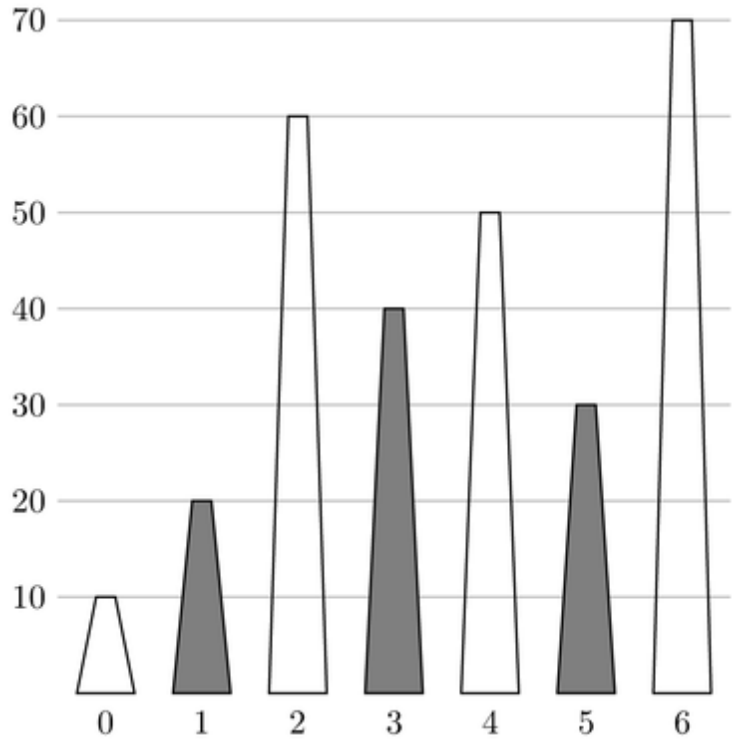
Example

Consider the following sequence of calls:

```
init(7, [10, 20, 60, 40, 50, 30, 70])
```

```
max_towers(1, 5, 10)
```

Pak Dengklek can lease towers 1, 3, and 5. The example is illustrated in the following picture, where shaded trapezoids represent leased towers.



Towers 3 and 5 can communicate using tower 4 as an intermediary, since $40 \leq 50 - 10$ and $30 \leq 50 - 10$. Towers 1 and 3 can communicate using tower 2 as an intermediary. Towers 1 and 5 can communicate using tower 3 as an intermediary. There is no way to lease more than 3 towers, therefore the procedure should return 3.

```
max_towers(2, 2, 100)
```

There is only 1 tower in the range, thus Pak Dengklek can only lease 1 tower. Therefore the procedure should return 1.

```
max_towers(0, 6, 17)
```

Pak Dengklek can lease towers 1 and 3. Towers 1 and 3 can communicate using tower 2 as an intermediary, since $20 \leq 60 - 17$ and $40 \leq 60 - 17$. There is no way to lease more than 2 towers, therefore the procedure should return 2.

Constraints

- $1 \leq N \leq 100\,000$
- $1 \leq Q \leq 100\,000$
- $1 \leq H[i] \leq 10^9$ (for each i such that $0 \leq i \leq N - 1$)
- $H[i] \neq H[j]$ (for each i and j such that $0 \leq i < j \leq N - 1$)
- $0 \leq L \leq R \leq N - 1$
- $1 \leq D \leq 10^9$

Subtasks

1. (4 points) There exists a tower k ($0 \leq k \leq N - 1$) such that
    - for each i such that $0 \leq i \leq k - 1$: $H[i] < H[i + 1]$, and
    - for each i such that $k \leq i \leq N - 2$: $H[i] > H[i + 1]$.
2. (11 points) $Q = 1$, $N \leq 2000$
3. (12 points) $Q = 1$
4. (14 points) $D = 1$
5. (17 points) $L = 0$, $R = N - 1$
6. (19 points) The value of D is the same across all `max_towers` calls.
7. (23 points) No additional constraints.

Sample Grader

The sample grader reads the input in the following format:

- line 1: N Q
- line 2: $H[0]$ $H[1]$ $\ldots$ $H[N - 1]$
- line $3 + j$ ($0 \leq j \leq Q - 1$): L R D for question j

The sample grader prints your answers in the following format:

- line $1 + j$ ($0 \leq j \leq Q - 1$): the return value of `max_towers` for question j

Rarest Insects

There are N insects, indexed from 0 to $N-1$, running around Pak Blangkon's house. Each insect has a **type**, which is an integer between 0 and 10^9 inclusive. Multiple insects may have the same type.

Suppose insects are grouped by type. We define the cardinality of the **most frequent** insect type as the number of insects in a group with the most number of insects. Similarly, the cardinality of the **rarest** insect type is the number of insects in a group with the least number of insects.

For example, suppose that there are 11 insects, whose types are $[5, 7, 9, 11, 11, 5, 0, 11, 9, 100, 9]$. In this case, the cardinality of the **most frequent** insect type is 3. The groups with the most number of insects are type 9 and type 11, each consisting of 3 insects. The cardinality of the **rarest** insect type is 1. The groups with the least number of insects are type 7, type 0, and type 100, each consisting of 1 insect.

Pak Blangkon does not know the type of any insect. He has a machine with a single button that can provide some information about the types of the insects. Initially, the machine is empty. To use the machine, three types of operations can be performed:

1. Move an insect to inside the machine.
2. Move an insect to outside the machine.
3. Press the button on the machine.

Each type of operation can be performed at most $40\ 000$ times.

Whenever the button is pressed, the machine reports the cardinality of the **most frequent** insect type, considering only insects inside the machine.

Your task is to determine the cardinality of the **rarest** insect type among all N insects in Pak Blangkon's house by using the machine. Additionally, in some subtasks, your score depends on the maximum number of operations of a given type that are performed (see Subtasks section for details).

Implementation Details

You should implement the following procedure:

```
int min_cardinality(int N)
```

- N: the number of insects.
- This procedure should return the cardinality of the **rarest** insect type among all N insects in Pak Blangkon's house.
- This procedure is called exactly once.

The above procedure can make calls to the following procedures:

```
void move_inside(int i)
```

- i: the index of the insect to be moved inside the machine. The value of i must be between 0 and $N - 1$ inclusive.
- If this insect is already inside the machine, the call has no effect on the set of insects in the machine. However, it is still counted as a separate call.
- This procedure can be called at most $40\ 000$ times.

```
void move_outside(int i)
```

- i: the index of the insect to be moved outside the machine. The value of i must be between 0 and $N - 1$ inclusive.
- If this insect is already outside the machine, the call has no effect on the set of insects in the machine. However, it is still counted as a separate call.
- This procedure can be called at most $40\ 000$ times.

```
int press_button()
```

- This procedure returns the cardinality of the **most frequent** insect type, considering only insects inside the machine.
- This procedure can be called at most $40\ 000$ times.
- The grader is **not adaptive**. That is, the types of all N insects are fixed before `min_cardinality` is called.

Example

Consider a scenario in which there are 6 insects of types $[5, 8, 9, 5, 9, 9]$ respectively. The procedure `min_cardinality` is called in the following way:

```
min_cardinality(6)
```

The procedure may call `move_inside`, `move_outside`, and `press_button` as follows.

Call	Return value	Insects in the machine	Types of insects in the machine
		$\{\}$	$[]$
`move_inside(0)`		$\{0\}$	$[5]$
`press_button()`	1	$\{0\}$	$[5]$
`move_inside(1)`		$\{0, 1\}$	$[5, 8]$
`press_button()`	1	$\{0, 1\}$	$[5, 8]$
`move_inside(3)`		$\{0, 1, 3\}$	$[5, 8, 5]$
`press_button()`	2	$\{0, 1, 3\}$	$[5, 8, 5]$
`move_inside(2)`		$\{0, 1, 2, 3\}$	$[5, 8, 9, 5]$
`move_inside(4)`		$\{0, 1, 2, 3, 4\}$	$[5, 8, 9, 5, 9]$
`move_inside(5)`		$\{0, 1, 2, 3, 4, 5\}$	$[5, 8, 9, 5, 9, 9]$
`press_button()`	3	$\{0, 1, 2, 3, 4, 5\}$	$[5, 8, 9, 5, 9, 9]$
`move_inside(5)`		$\{0, 1, 2, 3, 4, 5\}$	$[5, 8, 9, 5, 9, 9]$
`press_button()`	3	$\{0, 1, 2, 3, 4, 5\}$	$[5, 8, 9, 5, 9, 9]$
`move_outside(5)`		$\{0, 1, 2, 3, 4\}$	$[5, 8, 9, 5, 9]$
`press_button()`	2	$\{0, 1, 2, 3, 4\}$	$[5, 8, 9, 5, 9]$

At this point, there is sufficient information to conclude that the cardinality of the rarest insect type is 1. Therefore, the procedure `min_cardinality` should return 1.

In this example, `move_inside` is called 7 times, `move_outside` is called 1 time, and `press_button` is called 6 times.

Constraints

- $2 \leq N \leq 2000$

Subtasks

1. (10 points) $N \leq 200$
2. (15 points) $N \leq 1000$
3. (75 points) No additional constraints.

If in any of the test cases, the calls to the procedures `move_inside`, `move_outside`, or `press_button` do not conform to the constraints described in Implementation Details, or the

return value of `min_cardinality` is incorrect, the score of your solution for that subtask will be 0.

Let q be the **maximum** of the following three values: the number of calls to `move_inside`, the number of calls to `move_outside`, and the number of calls to `press_button`.

In subtask 3, you can obtain a partial score. Let m be the maximum value of $\frac{q}{N}$ across all test cases in this subtask. Your score for this subtask is calculated according to the following table:

Condition	Points
$20 < m$	0 (reported as "`Output isn't correct`" in CMS)
$6 < m \leq 20$	$\frac{225}{m-2}$
$3 < m \leq 6$	$81 - \frac{2}{3}m^2$
$m \leq 3$	75

Sample Grader

Let T be an array of N integers where $T[i]$ is the type of insect i.

The sample grader reads the input in the following format:

- line 1: N
- line 2: $T[0]\ T[1]\ \ldots\ T[N-1]$

If the sample grader detects a protocol violation, the output of the sample grader is `Protocol Violation: <MSG>`, where `<MSG>` is one of the following:

- `invalid parameter`: in a call to `move_inside` or `move_outside`, the value of i is not between 0 and $N-1$ inclusive.
- `too many calls`: the number of calls to **any** of `move_inside`, `move_outside`, or `press_button` exceeds $40\,000$.

Otherwise, the output of the sample grader is in the following format:

- line 1: the return value of `min_cardinality`
- line 2: q

Distributing Candies

Aunty Khong is preparing n boxes of candies for students from a nearby school. The boxes are numbered from 0 to $n-1$ and are initially empty. Box i ($0 \le i \le n-1$) has a capacity of $c[i]$ candies.

Aunty Khong spends q days preparing the boxes. On day j ($0 \le j \le q-1$), she performs an action specified by three integers $l[j]$, $r[j]$ and $v[j]$ where $0 \le l[j] \le r[j] \le n-1$ and $v[j] \neq 0$. For each box k satisfying $l[j] \le k \le r[j]$:

- If $v[j] > 0$, Aunty Khong adds candies to box k, one by one, until she has added exactly $v[j]$ candies or the box becomes full. In other words, if the box had p candies before the action, it will have $\min(c[k], p + v[j])$ candies after the action.
- If $v[j] < 0$, Aunty Khong removes candies from box k, one by one, until she has removed exactly $-v[j]$ candies or the box becomes empty. In other words, if the box had p candies before the action, it will have $\max(0, p + v[j])$ candies after the action.

Your task is to determine the number of candies in each box after the q days.

Implementation Details

You should implement the following procedure:

```
int[] distribute_candies(int[] c, int[] l, int[] r, int[] v)
```

- c: an array of length n. For $0 \le i \le n-1$, $c[i]$ denotes the capacity of box i.
- l, r and v: three arrays of length q. On day j, for $0 \le j \le q-1$, Aunty Khong performs an action specified by integers $l[j]$, $r[j]$ and $v[j]$, as described above.
- This procedure should return an array of length n. Denote the array by s. For $0 \le i \le n-1$, $s[i]$ should be the number of candies in box i after the q days.

Examples

Example 1

Consider the following call:

```
distribute_candies([10, 15, 13], [0, 0], [2, 1], [20, -11])
```

This means that box 0 has a capacity of 10 candies, box 1 has a capacity of 15 candies, and box 2 has a capacity of 13 candies.

At the end of day 0, box 0 has $\min(c[0], 0 + v[0]) = 10$ candies, box 1 has $\min(c[1], 0 + v[0]) = 15$ candies and box 2 has $\min(c[2], 0 + v[0]) = 13$ candies.

At the end of day 1, box 0 has $\max(0, 10 + v[1]) = 0$ candies, box 1 has $\max(0, 15 + v[1]) = 4$ candies. Since $2 > r[1]$, there is no change in the number of candies in box 2. The number of candies at the end of each day are summarized below:

Day	Box 0	Box 1	Box 2
0	10	15	13
1	0	4	13

As such, the procedure should return $[0, 4, 13]$.

Constraints

- $1 \leq n \leq 200\,000$
- $1 \leq q \leq 200\,000$
- $1 \leq c[i] \leq 10^9$ (for all $0 \leq i \leq n - 1$)
- $0 \leq l[j] \leq r[j] \leq n - 1$ (for all $0 \leq j \leq q - 1$)
- $-10^9 \leq v[j] \leq 10^9, v[j] \neq 0$ (for all $0 \leq j \leq q - 1$)

Subtasks

1. (3 points) $n, q \leq 2000$
2. (8 points) $v[j] > 0$ (for all $0 \leq j \leq q - 1$)
3. (27 points) $c[0] = c[1] = \ldots = c[n - 1]$
4. (29 points) $l[j] = 0$ and $r[j] = n - 1$ (for all $0 \leq j \leq q - 1$)
5. (33 points) No additional constraints.

Sample Grader

The sample grader reads in the input in the following format:

- line 1: n
- line 2: $c[0]\ c[1]\ \ldots\ c[n - 1]$
- line 3: q
- line $4 + j$ ($0 \leq j \leq q - 1$): $l[j]\ r[j]\ v[j]$

The sample grader prints your answers in the following format:

- line 1: $s[0]\ s[1]\ \ldots\ s[n - 1]$

Keys

Timothy the architect has designed a new escape game. In this game, there are n rooms numbered from 0 to $n-1$. Initially, each room contains exactly one key. Each key has a type, which is an integer between 0 and $n-1$, inclusive. The type of the key in room i ($0 \leq i \leq n-1$) is $r[i]$. Note that multiple rooms may contain keys of the same type, i.e., the values $r[i]$ are not necessarily distinct.

There are also m **bidirectional** connectors in the game, numbered from 0 to $m-1$. Connector j ($0 \leq j \leq m-1$) connects a pair of different rooms $u[j]$ and $v[j]$. A pair of rooms can be connected by multiple connectors.

The game is played by a single player who collects the keys and moves between the rooms by traversing the connectors. We say that the player **traverses** connector j when they use this connector to move from room $u[j]$ to room $v[j]$, or vice versa. The player can only traverse connector j if they have collected a key of type $c[j]$ before.

At any point during the game, the player is in some room x and can perform two types of actions:

- collect the key in room x, whose type is $r[x]$ (unless they have collected it already),
- traverse a connector j, where either $u[j] = x$ or $v[j] = x$, if the player has collected a key of type $c[j]$ beforehand. Note that the player **never** discards a key they have collected.

The player **starts** the game in some room s not carrying any keys. A room t is **reachable** from a room s, if the player who starts the game in room s can perform some sequence of actions described above, and reach room t.

For each room i ($0 \leq i \leq n-1$), denote the number of rooms reachable from room i as $p[i]$. Timothy would like to know the set of indices i that attain the minimum value of $p[i]$ across $0 \leq i \leq n-1$.

Implementation Details

You are to implement the following procedure:

```
int[] find_reachable(int[] r, int[] u, int[] v, int[] c)
```

- r: an array of length n. For each i ($0 \leq i \leq n-1$), the key in room i is of type $r[i]$.
- u, v: two arrays of length m. For each j ($0 \leq j \leq m-1$), connector j connects rooms $u[j]$ and $v[j]$.
- c: an array of length m. For each j ($0 \leq j \leq m-1$), the type of key needed to traverse connector j is $c[j]$.

- This procedure should return an array a of length n. For each $0 \leq i \leq n-1$, the value of $a[i]$ should be 1 if for every j such that $0 \leq j \leq n-1$, $p[i] \leq p[j]$. Otherwise, the value of $a[i]$ should be 0.

Examples

Example 1

Consider the following call:

```
find_reachable([0, 1, 1, 2],
               [0, 0, 1, 1, 3], [1, 2, 2, 3, 1], [0, 0, 1, 0, 2])
```

If the player starts the game in room 0, they can perform the following sequence of actions:

Current room	Action
0	Collect key of type 0
0	Traverse connector 0 to room 1
1	Collect key of type 1
1	Traverse connector 2 to room 2
2	Traverse connector 2 to room 1
1	Traverse connector 3 to room 3

Hence room 3 is reachable from room 0. Similarly, we can construct sequences showing that all rooms are reachable from room 0, which implies $p[0] = 4$. The table below shows the reachable rooms for all starting rooms:

Starting room i	Reachable rooms	$p[i]$
0	$[0, 1, 2, 3]$	4
1	$[1, 2]$	2
2	$[1, 2]$	2
3	$[1, 2, 3]$	3

The smallest value of $p[i]$ across all rooms is 2, and this is attained for $i = 1$ or $i = 2$. Therefore, this procedure should return $[0, 1, 1, 0]$.

Example 2

```
find_reachable([0, 1, 1, 2, 2, 1, 2],
               [0, 0, 1, 1, 2, 3, 3, 4, 4, 5],
               [1, 2, 2, 3, 3, 4, 5, 5, 6, 6],
               [0, 0, 1, 0, 0, 1, 2, 0, 2, 1])
```

The table below shows the reachable rooms:

Starting room i	Reachable rooms	$p[i]$
0	$[0, 1, 2, 3, 4, 5, 6]$	7
1	$[1, 2]$	2
2	$[1, 2]$	2
3	$[3, 4, 5, 6]$	4
4	$[4, 6]$	2
5	$[3, 4, 5, 6]$	4
6	$[4, 6]$	2

The smallest value of $p[i]$ across all rooms is 2, and this is attained for $i \in \{1, 2, 4, 6\}$. Therefore, this procedure should return $[0, 1, 1, 0, 1, 0, 1]$.

Example 3

```
find_reachable([0, 0, 0], [0], [1], [0])
```

The table below shows the reachable rooms:

Starting room i	Reachable rooms	$p[i]$
0	$[0, 1]$	2
1	$[0, 1]$	2
2	$[2]$	1

The smallest value of $p[i]$ across all rooms is 1, and this is attained when $i = 2$. Therefore, this procedure should return $[0, 0, 1]$.

Constraints

- $2 \leq n \leq 300\,000$
- $1 \leq m \leq 300\,000$
- $0 \leq r[i] \leq n - 1$ for all $0 \leq i \leq n - 1$
- $0 \leq u[j], v[j] \leq n - 1$ and $u[j] \neq v[j]$ for all $0 \leq j \leq m - 1$

- $0 \le c[j] \le n-1$ for all $0 \le j \le m-1$

Subtasks

1. (9 points) $c[j] = 0$ for all $0 \le j \le m-1$ and $n, m \le 200$
2. (11 points) $n, m \le 200$
3. (17 points) $n, m \le 2000$
4. (30 points) $c[j] \le 29$ (for all $0 \le j \le m-1$) and $r[i] \le 29$ (for all $0 \le i \le n-1$)
5. (33 points) No additional constraints.

Sample Grader

The sample grader reads the input in the following format:

- line 1: n m
- line 2: $r[0]$ $r[1]$ $\ldots$ $r[n-1]$
- line $3+j$ ($0 \le j \le m-1$): $u[j]$ $v[j]$ $c[j]$

The sample grader prints the return value of `find_reachable` in the following format:

- line 1: $a[0]$ $a[1]$ $\ldots$ $a[n-1]$

Fountain Parks

In a nearby park, there are n **fountains**, labeled from 0 to $n-1$. We model the fountains as points on a two-dimensional plane. Namely, fountain i ($0 \le i \le n-1$) is a point $(x[i], y[i])$ where $x[i]$ and $y[i]$ are **even integers**. The locations of the fountains are all distinct.

Timothy the architect has been hired to plan the construction of some **roads** and the placement of one **bench** per road. A road is a **horizontal** or **vertical** line segment of length 2, whose endpoints are two distinct fountains. The roads should be constructed such that one can travel between any two fountains by moving along roads. Initially, there are no roads in the park.

For each road, **exactly** one bench needs to be placed in the park and **assigned to** (i.e., face) that road. Each bench must be placed at some point (a, b) such that a and b are **odd integers**. The locations of the benches must be all **distinct**. A bench at (a, b) can only be assigned to a road if **both** of the road's endpoints are among $(a-1, b-1)$, $(a-1, b+1)$, $(a+1, b-1)$ and $(a+1, b+1)$. For example, the bench at $(3, 3)$ can only be assigned to a road, which is one of the four line segments $(2, 2)$ – $(2, 4)$, $(2, 4)$ – $(4, 4)$, $(4, 4)$ – $(4, 2)$, $(4, 2)$ – $(2, 2)$.

Help Timothy determine if it is possible to construct roads, and place and assign benches satisfying all conditions given above, and if so, provide him with a feasible solution. If there are multiple feasible solutions that satisfy all conditions, you can report any of them.

Implementation Details

You should implement the following procedure:

```
int construct_roads(int[] x, int[] y)
```

- x, y: two arrays of length n. For each i ($0 \le i \le n-1$), fountain i is a point $(x[i], y[i])$, where $x[i]$ and $y[i]$ are even integers.
- If a construction is possible, this procedure should make exactly one call to `build` (see below) to report a solution, following which it should return 1.
- Otherwise, the procedure should return 0 without making any calls to `build`.
- This procedure is called exactly once.

Your implementation can call the following procedure to provide a feasible construction of roads and a placement of benches:

```
void build(int[] u, int[] v, int[] a, int[] b)
```

- Let m be the total number of roads in the construction.

- u, v: two arrays of length m, representing the roads to be constructed. These roads are labeled from 0 to $m-1$. For each j ($0 \leq j \leq m-1$), road j connects fountains $u[j]$ and $v[j]$. Each road must be a horizontal or vertical line segment of length 2. Any two distinct roads can have at most one point in common (a fountain). Once the roads are constructed, it should be possible to travel between any two fountains by moving along roads.
- a, b: two arrays of length m, representing the benches. For each j ($0 \leq j \leq m-1$), a bench is placed at $(a[j], b[j])$, and is assigned to road j. No two distinct benches can have the same location.

Examples

Example 1

Consider the following call:

```
construct_roads([4, 4, 6, 4, 2], [4, 6, 4, 2, 4])
```

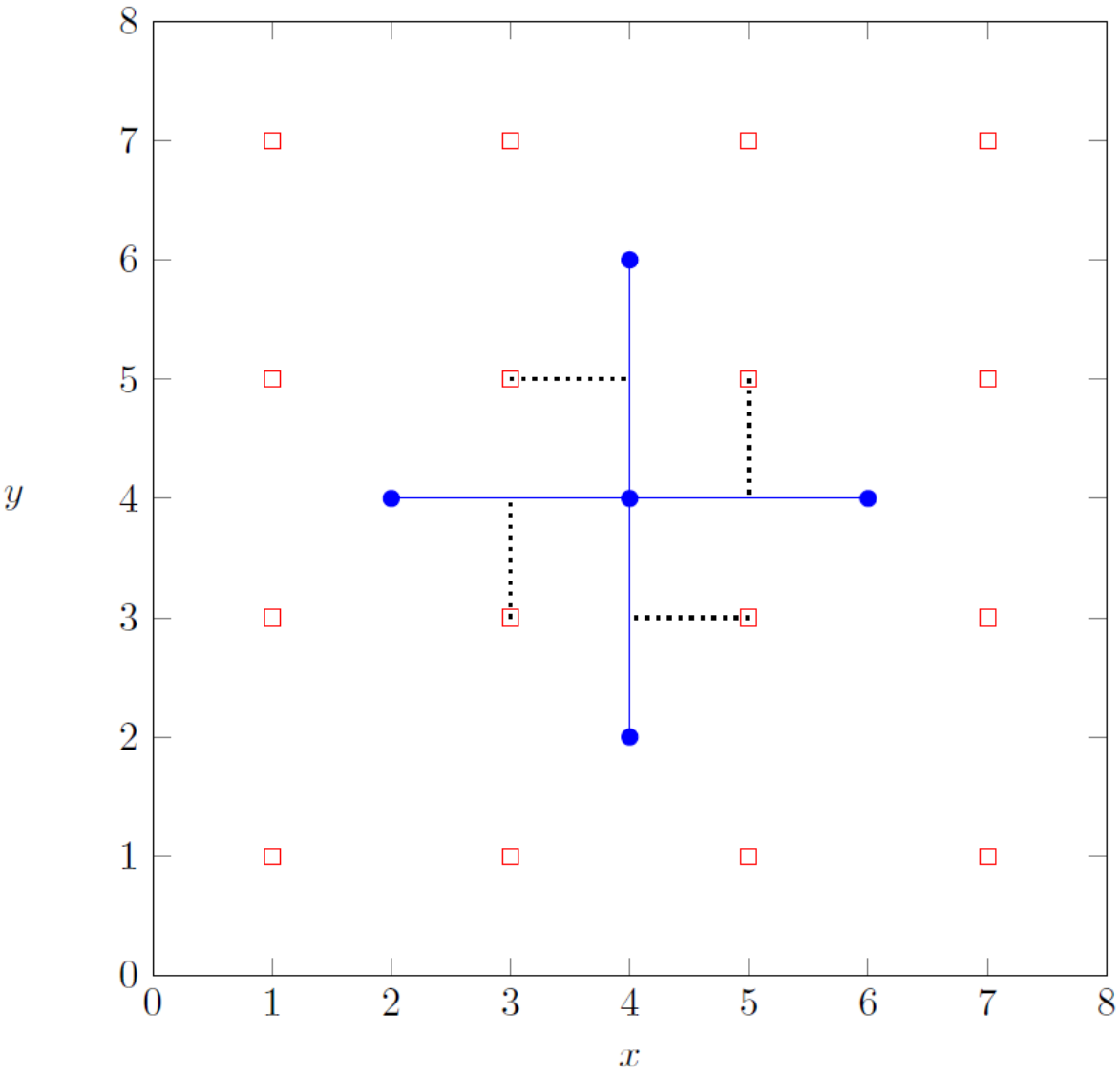
This means that there are 5 fountains:

- fountain 0 is located at $(4, 4)$,
- fountain 1 is located at $(4, 6)$,
- fountain 2 is located at $(6, 4)$,
- fountain 3 is located at $(4, 2)$,
- fountain 4 is located at $(2, 4)$.

It is possible to construct the following 4 roads, where each road connects two fountains, and place the corresponding benches:

Road label	Labels of the fountains the road connects	Location of the assigned bench
0	$0, 2$	$(5, 5)$
1	$0, 1$	$(3, 5)$
2	$3, 0$	$(5, 3)$
3	$4, 0$	$(3, 3)$

This solution corresponds to the following diagram:



To report this solution, `construct_roads` should make the following call:

- `build([0, 0, 3, 4], [2, 1, 0, 0], [5, 3, 5, 3], [5, 5, 3, 3])`

It should then return 1.

Note that in this case, there are multiple solutions that satisfy the requirements, all of which would be considered correct. For example, it is also correct to call `build([1, 2, 3, 4], [0, 0, 0, 0], [5, 5, 3, 3], [5, 3, 3, 5])` and then return 1.

Example 2

Consider the following call:

```
construct_roads([2, 4], [2, 6])
```

Fountain 0 is located at $(2, 2)$ and fountain 1 is located at $(4, 6)$. Since there is no way to construct roads that satisfy the requirements, `construct_roads` should return 0 without making any

call to `build`.

Constraints

- $1 \leq n \leq 200\,000$
- $2 \leq x[i], y[i] \leq 200\,000$ (for all $0 \leq i \leq n-1$)
- $x[i]$ and $y[i]$ are even integers (for all $0 \leq i \leq n-1$).
- No two fountains have the same location.

Subtasks

1. (5 points) $x[i] = 2$ (for all $0 \leq i \leq n-1$)
2. (10 points) $2 \leq x[i] \leq 4$ (for all $0 \leq i \leq n-1$)
3. (15 points) $2 \leq x[i] \leq 6$ (for all $0 \leq i \leq n-1$)
4. (20 points) There is at most one way of constructing the roads, such that one can travel between any two fountains by moving along roads.
5. (20 points) There do not exist four fountains that form the corners of a 2×2 square.
6. (30 points) No additional constraints.

Sample Grader

The sample grader reads the input in the following format:

- line 1: n
- line $2+i$ ($0 \leq i \leq n-1$): $x[i]$ $y[i]$

The output of the sample grader is in the following format:

- line 1: the return value of `construct_roads`

If the return value of `construct_roads` is 1 and `build(u, v, a, b)` is called, the grader then additionally prints:

- line 2: m
- line $3+j$ ($0 \leq j \leq m-1$): $u[j]$ $v[j]$ $a[j]$ $b[j]$

Mutating DNA

Grace is a biologist working in a bioinformatics firm in Singapore. As part of her job, she analyses the DNA sequences of various organisms. A DNA sequence is defined as a string consisting of characters "A", "T", and "C". Note that in this task DNA sequences **do not contain character "G".**

We define a mutation to be an operation on a DNA sequence where two elements of the sequence are swapped. For example a single mutation can transform "A**C**T**A**" into "A**A**T**C**" by swapping the highlighted characters "A" and "C".

The mutation distance between two sequences is the minimum number of mutations required to transform one sequence into the other, or -1 if it is not possible to transform one sequence into the other by using mutations.

Grace is analysing two DNA sequences a and b, both consisting of n elements with indices from 0 to $n-1$. Your task is to help Grace answer q questions of the form: what is the mutation distance between the substring $a[x..y]$ and the substring $b[x..y]$? Here, a substring $s[x..y]$ of a DNA sequence s is defined to be a sequence of consecutive characters of s, whose indices are x to y inclusive. In other words, $s[x..y]$ is the sequence $s[x]s[x+1]\ \ldots\ s[y]$.

Implementation details

You should implement the following procedures:

```
void init(string a, string b)
```

- a, b: strings of length n, describing the two DNA sequences to be analysed.
- This procedure is called exactly once, before any calls to `get_distance`.

```
int get_distance(int x, int y)
```

- x, y: starting and ending indices of the substrings to be analysed.
- The procedure should return the mutation distance between substrings $a[x..y]$ and $b[x..y]$.
- This procedure is called exactly q times.

Example

Consider the following call:

```
init("ATACAT", "ACTATA")
```

Let's say the grader calls `get_distance(1, 3)`. This call should return the mutation distance between $a[1..3]$ and $b[1..3]$, that is, the sequences "TAC" and "CTA". "TAC" can be transformed into "CTA" via 2 mutations: **TA**C $\rightarrow$ **CA**T, followed by C**AT** $\rightarrow$ C**TA**, and the transformation is impossible with fewer than 2 mutations.

Therefore, this call should return 2.

Let's say the grader calls `get_distance(4, 5)`. This call should return the mutation distance between sequences "AT" and "TA". "AT" can be transformed into "TA" through a single mutation, and clearly at least one mutation is required.

Therefore, this call should return 1.

Finally, let's say the grader calls `get_distance(3, 5)`. Since there is **no way** for the sequence "CAT" to be transformed into "ATA" via any sequence of mutations, this call should return -1.

Constraints

- $1 \leq n, q \leq 100\ 000$
- $0 \leq x \leq y \leq n - 1$
- Each character of a and b is one of "A", "T", and "C".

Subtasks

1. (21 points) $y - x \leq 2$
2. (22 points) $q \leq 500,\ y - x \leq 1000$, each character of a and b is either "A" or "T".
3. (13 points) each character of a and b is either "A" or "T".
4. (28 points) $q \leq 500,\ y - x \leq 1000$
5. (16 points) No additional constraints.

Sample grader

The sample grader reads the input in the following format:

- line 1: n q
- line 2: a
- line 3: b
- line $4 + i$ ($0 \leq i \leq q - 1$): x y for the i-th call to `get_distance`.

The sample grader prints your answers in the following format:

- line $1 + i$ ($0 \leq i \leq q - 1$) : the return value of the i-th call to `get_distance`.

Dungeons Game

Robert is designing a new computer game. The game involves one hero, n opponents and $n+1$ dungeons. The opponents are numbered from 0 to $n-1$ and the dungeons are numbered from 0 to n. Opponent i ($0 \leq i \leq n-1$) is located in dungeon i and has strength $s[i]$. There is no opponent in dungeon n.

The hero starts off entering dungeon x, with strength z. Every time the hero enters any dungeon i ($0 \leq i \leq n-1$), they confront opponent i, and one of the following occurs:

- If the hero's strength is greater than or equal to the opponent's strength $s[i]$, the hero wins. This causes the hero's strength to **increase** by $s[i]$ ($s[i] \geq 1$). In this case the hero enters dungeon $w[i]$ next ($w[i] > i$).
- Otherwise, the hero loses. This causes the hero's strength to **increase** by $p[i]$ ($p[i] \geq 1$). In this case the hero enters dungeon $l[i]$ next.

Note $p[i]$ may be less than, equal to, or greater than $s[i]$. Also, $l[i]$ may be less than, equal to, or greater than i. Regardless of the outcome of the confrontation, the opponent remains in dungeon i and maintains strength $s[i]$.

The game ends when the hero enters dungeon n. One can show that the game ends after a finite number of confrontations, regardless of the hero's starting dungeon and strength.

Robert asked you to test his game by running q simulations. For each simulation, Robert defines a starting dungeon x and starting strength z. Your task is to find out, for each simulation, the hero's strength when the game ends.

Implementation details

You should implement the following procedures:

```
void init(int n, int[] s, int[] p, int[] w, int[] l)
```

- n: number of opponents.
- s, p, w, l: arrays of length n. For $0 \leq i \leq n-1$:
  - $s[i]$ is the strength of the opponent i. It is also the strength gained by the hero after winning against opponent i.
  - $p[i]$ is the strength gained by the hero after losing against opponent i.
  - $w[i]$ is the dungeon the hero enters after winning against opponent i.
  - $l[i]$ is the dungeon the hero enters after losing against opponent i.
- This procedure is called exactly once, before any calls to `simulate` (see below).

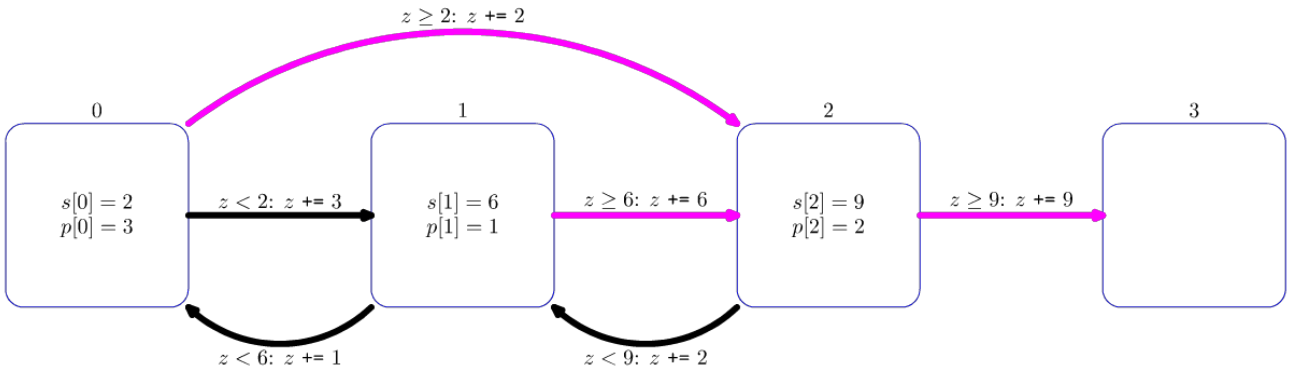
```
int64 simulate(int x, int z)
```

- x: the dungeon the hero enters first.
- z: the hero's starting strength.
- This procedure should return the hero's strength when the game ends, assuming the hero starts the game by entering dungeon x, having strength z.
- The procedure is called exactly q times.

Example

Consider the following call:

```
init(3, [2, 6, 9], [3, 1, 2], [2, 2, 3], [1, 0, 1])
```



The diagram above illustrates this call. Each square shows a dungeon. For dungeons 0, 1 and 2, the values $s[i]$ and $p[i]$ are indicated inside the squares. Magenta arrows indicate where the hero moves after winning a confrontation, while black arrows indicate where the hero moves after losing.

Let's say the grader calls `simulate(0, 1)`.

The game proceeds as follows:

Dungeon	Hero's strength before confrontation	Result
0	1	Lose
1	4	Lose
0	5	Win
2	7	Lose
1	9	Win
2	15	Win
3	24	Game ends

As such, the procedure should return 24.

Let's say the grader calls `simulate(2, 3)`.

The game proceeds as follows:

Dungeon	Hero's strength before confrontation	Result
2	3	Lose
1	5	Lose
0	6	Win
2	8	Lose
1	10	Win
2	16	Win
3	25	Game ends

As such, the procedure should return 25.

Constraints

- $1 \leq n \leq 400\ 000$
- $1 \leq q \leq 50\ 000$
- $1 \leq s[i], p[i] \leq 10^7$ (for all $0 \leq i \leq n-1$)
- $0 \leq l[i], w[i] \leq n$ (for all $0 \leq i \leq n-1$)
- $w[i] > i$ (for all $0 \leq i \leq n-1$)
- $0 \leq x \leq n-1$
- $1 \leq z \leq 10^7$

Subtasks

1. (11 points) $n \leq 50\ 000$, $q \leq 100$, $s[i], p[i] \leq 10\ 000$ (for all $0 \leq i \leq n-1$)
2. (26 points) $s[i] = p[i]$ (for all $0 \leq i \leq n-1$)
3. (13 points) $n \leq 50\ 000$, all opponents have the same strength, in other words, $s[i] = s[j]$ for all $0 \leq i, j \leq n-1$.
4. (12 points) $n \leq 50\ 000$, there are at most 5 distinct values among all values of $s[i]$.
5. (27 points) $n \leq 50\ 000$
6. (11 points) No additional constraints.

Sample grader

The sample grader reads the input in the following format:

- line 1: n q
- line 2: $s[0]$ $s[1]$ $\ldots$ $s[n-1]$
- line 3: $p[0]$ $p[1]$ $\ldots$ $p[n-1]$

- line 4: $w[0]$ $w[1]$ $\ldots$ $w[n-1]$
- line 5: $l[0]$ $l[1]$ $\ldots$ $l[n-1]$
- line $6+i$ ($0 \le i \le q-1$): x z for the i-th call to `simulate`.

The sample grader prints your answers in the following format:

- line $1+i$ ($0 \le i \le q-1$) : the return value of the i-th call to `simulate`.

Comparing Plants (plants)

Hazel the botanist visited a special exhibition in the Singapore Botanical Gardens. In this exhibition, n plants of **distinct heights** are placed in a circle. These plants are labelled from 0 to $n-1$ in clockwise order, with plant $n-1$ beside plant 0.

For each plant i ($0 \leq i \leq n-1$), Hazel compared plant i to each of the next $k-1$ plants in clockwise order, and wrote down the number $r[i]$ denoting how many of these $k-1$ plants are taller than plant i. Thus, each value $r[i]$ depends on the relative heights of some k consecutive plants.

For example, suppose $n = 5$, $k = 3$ and $i = 3$. The next $k-1 = 2$ plants in clockwise order from plant $i = 3$ would be plant 4 and plant 0. If plant 4 was taller than plant 3 and plant 0 was shorter than plant 3, Hazel would write down $r[3] = 1$.

You may assume that Hazel recorded the values $r[i]$ correctly. Thus, there is at least one configuration of distinct heights of plants consistent with these values.

You were asked to compare the heights of q pairs of plants. Sadly, you do not have access to the exhibition. Your only source of information is Hazel's notebook with the value k and the sequence of values $r[0], \ldots, r[n-1]$.

For each pair of different plants x and y that need to be compared, determine which of the three following situations occurs:

- Plant x is definitely taller than plant y: in any configuration of distinct heights $h[0], \ldots, h[n-1]$ consistent with the array r we have $h[x] > h[y]$.
- Plant x is definitely shorter than plant y: in any configuration of distinct heights $h[0], \ldots, h[n-1]$ consistent with the array r we have $h[x] < h[y]$.
- The comparison is inconclusive: neither of the previous two cases applies.

Implementation details

You should implement the following procedures:

```
void init(int k, int[] r)
```

- k: the number of consecutive plants whose heights determine each individual value $r[i]$.
- r: an array of size n, where $r[i]$ is the number of plants taller than plant i among the next $k-1$ plants in clockwise order.
- This procedure is called exactly once, before any calls to `compare_plants`.

```
int compare_plants(int x, int y)
```

- x, y: labels of the plants to be compared.
- This procedure should return:
  - 1 if plant x is definitely taller than plant y,
  - -1 if plant x is definitely shorter than plant y,
  - 0 if the comparison is inconclusive.
- This procedure is called exactly q times.

Examples

Example 1

Consider the following call:

```
init(3, [0, 1, 1, 2])
```

Let's say the grader calls `compare_plants(0, 2)`. Since $r[0] = 0$ we can immediately infer that plant 2 is not taller than plant 0. Therefore, the call should return 1.

Let's say the grader calls `compare_plants(1, 2)` next. For all possible configurations of heights that fit the constraints above, plant 1 is shorter than plant 2. Therefore, the call should return -1.

Example 2

Consider the following call:

```
init(2, [0, 1, 0, 1])
```

Let's say the grader calls `compare_plants(0, 3)`. Since $r[3] = 1$, we know that plant 0 is taller than plant 3. Therefore, the call should return 1.

Let's say the grader calls `compare_plants(1, 3)` next. Two configurations of heights $[3, 1, 4, 2]$ and $[3, 2, 4, 1]$ are both consistent with Hazel's measurements. Since plant 1 is shorter than plant 3 in one configuration and taller than plant 3 in the other, this call should return 0.

Constraints

- $2 \leq k \leq n \leq 200\,000$
- $1 \leq q \leq 200\,000$
- $0 \leq r[i] \leq k - 1$ (for all $0 \leq i \leq n - 1$)
- $0 \leq x < y \leq n - 1$
- There exists one or more configurations of **distinct heights** of plants consistent with the array

r.

Subtasks

1. (5 points) $k = 2$
2. (14 points) $n \leq 5000$, $2 \cdot k > n$
3. (13 points) $2 \cdot k > n$
4. (17 points) The correct answer to each call of `compare_plants` is 1 or -1.
5. (11 points) $n \leq 300, q \leq \frac{n \cdot (n-1)}{2}$
6. (15 points) $x = 0$ for each call of `compare_plants`.
7. (25 points) No additional constraints.

Sample grader

The sample grader reads the input in the following format:

- line 1: n k q
- line 2: $r[0]$ $r[1]$ $\ldots$ $r[n-1]$
- line $3 + i$ $(0 \leq i \leq q - 1)$: x y for the i-th call to `compare_plants`

The sample grader prints your answers in the following format:

- line $1 + i$ $(0 \leq i \leq q - 1)$: return value of the i-th call to `compare_plants`.

Connecting Supertrees (supertrees)

Gardens by the Bay is a large nature park in Singapore. In the park there are n towers, known as supertrees. These towers are labelled 0 to $n-1$. We would like to construct a set of **zero or more** bridges. Each bridge connects a pair of distinct towers and may be traversed in **either** direction. No two bridges should connect the same pair of towers.

A path from tower x to tower y is a sequence of one or more towers such that:

- the first element of the sequence is x,
- the last element of the sequence is y,
- all elements of the sequence are **distinct**, and
- each two consecutive elements (towers) in the sequence are connected by a bridge.

Note that by definition there is exactly one path from a tower to itself and the number of different paths from tower i to tower j is the same as the number of different paths from tower j to tower i.

The lead architect in charge of the design wishes for the bridges to be built such that for all $0 \leq i, j \leq n-1$ there are exactly $p[i][j]$ different paths from tower i to tower j, where $0 \leq p[i][j] \leq 3$.

Construct a set of bridges that satisfy the architect's requirements, or determine that it is impossible.

Implementation details

You should implement the following procedure:

```
int construct(int[][] p)
```

- p: an $n \times n$ array representing the architect's requirements.
- If a construction is possible, this procedure should make exactly one call to `build` (see below) to report the construction, following which it should return 1.
- Otherwise, the procedure should return 0 without making any calls to `build`.
- This procedure is called exactly once.

The procedure `build` is defined as follows:

```
void build(int[][] b)
```

- b: an $n \times n$ array, with $b[i][j] = 1$ if there is a bridge connecting tower i and tower j, or

$b[i][j] = 0$ otherwise.
- Note that the array must satisfy $b[i][j] = b[j][i]$ for all $0 \leq i, j \leq n-1$ and $b[i][i] = 0$ for all $0 \leq i \leq n-1$.

Examples

Example 1

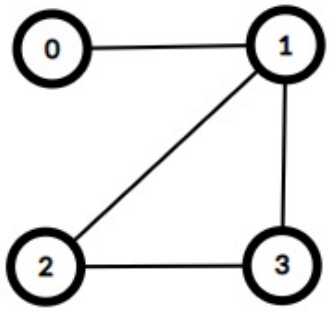
Consider the following call:

```
construct([[1, 1, 2, 2], [1, 1, 2, 2], [2, 2, 1, 2], [2, 2, 2, 1]])
```

This means that there should be exactly one path from tower 0 to tower 1. For all other pairs of towers (x, y), such that $0 \leq x < y \leq 3$, there should be exactly two paths from tower x to tower y. This can be achieved with 4 bridges, connecting pairs of towers $(0, 1)$, $(1, 2)$, $(1, 3)$ and $(2, 3)$.

To report this solution, the `construct` procedure should make the following call:

- `build([[0, 1, 0, 0], [1, 0, 1, 1], [0, 1, 0, 1], [0, 1, 1, 0]])`



It should then return 1.

In this case, there are multiple constructions that fit the requirements, all of which would be considered correct.

Example 2

Consider the following call:

```
construct([[1, 0], [0, 1]])
```

This means that there should be no way to travel between the two towers. This can only be satisfied by having no bridges.

Therefore, the `construct` procedure should make the following call:

- `build([[0, 0], [0, 0]])`

After which, the `construct` procedure should return 1.

Example 3

Consider the following call:

```
construct([[1, 3], [3, 1]])
```

This means that there should be exactly 3 paths from tower 0 to tower 1. This set of requirements cannot be satisfied. As such, the `construct` procedure should return 0 without making any call to `build`.

Constraints

- $1 \leq n \leq 1000$
- $p[i][i] = 1$ (for all $0 \leq i \leq n-1$)
- $p[i][j] = p[j][i]$ (for all $0 \leq i, j \leq n-1$)
- $0 \leq p[i][j] \leq 3$ (for all $0 \leq i, j \leq n-1$)

Subtasks

1. (11 points) $p[i][j] = 1$ (for all $0 \leq i, j \leq n-1$)
2. (10 points) $p[i][j] = 0$ or 1 (for all $0 \leq i, j \leq n-1$)
3. (19 points) $p[i][j] = 0$ or 2 (for all $i \neq j$, $0 \leq i, j \leq n-1$)
4. (35 points) $0 \leq p[i][j] \leq 2$ (for all $0 \leq i, j \leq n-1$) and there is at least one construction satisfying the requirements.
5. (21 points) $0 \leq p[i][j] \leq 2$ (for all $0 \leq i, j \leq n-1$)
6. (4 points) No additional constraints.

Sample grader

The sample grader reads the input in the following format:

- line 1: n
- line $2+i$ ($0 \leq i \leq n-1$): $p[i][0]$ $p[i][1]$ $\ldots$ $p[i][n-1]$

The output of the sample grader is in the following format:

- line 1: the return value of `construct`.

If the return value of `construct` is 1, the sample grader additionally prints:

- line $2+i$ ($0 \leq i \leq n-1$): $b[i][0]$ $b[i][1]$ $\ldots$ $b[i][n-1]$

Carnival Tickets (tickets)

Ringo is at a carnival in Singapore. He has some prize tickets in his bag, which he would like to use at the prize game stall. Each ticket comes in one of n colours and has a non-negative integer printed on it. The integers printed on different tickets might be the same. Due to a quirk in the carnival rules, n is guaranteed to be **even**.

Ringo has m tickets of each colour in his bag, that is a total of $n \cdot m$ tickets. The ticket j of the colour i has the integer $x[i][j]$ printed on it ($0 \leq i \leq n-1$ and $0 \leq j \leq m-1$).

The prize game is played in k rounds, numbered from 0 to $k-1$. Each round is played in the following order:

- From his bag, Ringo selects a **set** of n tickets, one ticket from each colour. He then gives the set to the game master.
- The game master notes down the integers $a[0],\ a[1]\ \ \ldots\ \ a[n-1]$ printed on the tickets of the set. The order of these n integers is not important.
- The game master pulls out a special card from a lucky draw box and notes down the integer b printed on that card.
- The game master calculates the absolute differences between $a[i]$ and b for each i from 0 to $n-1$. Let S be the sum of these absolute differences.
- For this round, the game master gives Ringo a prize with a value equal to S.
- The tickets in the set are discarded and cannot be used in future rounds.

The remaining tickets in Ringo's bag after k rounds of the game are discarded.

By watching closely, Ringo realized that the prize game is rigged! There is actually a printer inside the lucky draw box. In each round, the game master finds an integer b that minimizes the value of the prize of that round. The value chosen by the game master is printed on the special card for that round.

Having all this information, Ringo would like to allocate tickets to the rounds of the game. That is, he wants to select the ticket set to use in each round in order to maximize the total value of the prizes.

Implementation details

You should implement the following procedure:

```
int64 find_maximum(int k, int[][] x)
```

- k: the number of rounds.

- x: an $n \times m$ array describing the integers on each ticket. Tickets of each color are sorted in non-decreasing order of their integers.
- This procedure is called exactly once.
- This procedure should make exactly one call to `allocate_tickets` (see below), describing k ticket sets, one for each round. The allocation should maximize the total value of the prizes.
- This procedure should return the maximum total value of the prizes.

The procedure `allocate_tickets` is defined as follows:

```
void allocate_tickets(int[][] s)
```

- s: an $n \times m$ array. The value of $s[i][j]$ should be r if the ticket j of the colour i is used in the set of round r of the game, or -1 if it is not used at all.
- For each $0 \leq i \leq n-1$, among $s[i][0], s[i][1], \ldots, s[i][m-1]$ each value $0, 1, 2, \ldots, k-1$ must occur exactly once, and all other entries must be -1.
- If there are multiple allocations resulting in the maximum total prize value, it is allowed to report any of them.

Examples

Example 1

Consider the following call:

```
find_maximum(2, [[0, 2, 5],[1, 1, 3]])
```

This means that:

- there are $k = 2$ rounds;
- the integers printed on the tickets of colour 0 are 0, 2 and 5, respectively;
- the integers printed on the tickets of colour 1 are 1, 1 and 3, respectively.

A possible allocation that gives the maximum total prize value is:

- In round 0, Ringo picks ticket 0 of colour 0 (with the integer 0) and ticket 2 of colour 1 (with the integer 3). The lowest possible value of the prize in this round is 3. E.g., the game master may choose $b = 1$: $|1 - 0| + |1 - 3| = 1 + 2 = 3$.
- In round 1, Ringo picks ticket 2 of colour 0 (with the integer 5) and ticket 1 of colour 1 (with the integer 1). The lowest possible value of the prize in this round is 4. E.g., the game master may choose $b = 3$: $|3 - 1| + |3 - 5| = 2 + 2 = 4$.
- Therefore, the total value of the prizes would be $3 + 4 = 7$.

To report this allocation, the procedure `find_maximum` should make the following call to `allocate_tickets`:

- `allocate_tickets([[0, -1, 1], [-1, 1, 0]])`

Finally, the procedure `find_maximum` should return 7.

Example 2

Consider the following call:

```
find_maximum(1, [[5, 9], [1, 4], [3, 6], [2, 7]])
```

This means that:

- there is only one round,
- the integers printed on the tickets of colour 0 are 5 and 9, respectively;
- the integers printed on the tickets of colour 1 are 1 and 4, respectively;
- the integers printed on the tickets of colour 2 are 3 and 6, respectively;
- the integers printed on the tickets of colour 3 are 2 and 7, respectively.

A possible allocation that gives the maximum total prize value is:

- In round 0, Ringo picks ticket 1 of colour 0 (with the integer 9), ticket 0 of colour 1 (with the integer 1), ticket 0 of colour 2 (with the integer 3), and ticket 1 of colour 3 (with the integer 7). The lowest possible value of the prize in this round is 12, when the game master chooses $b = 3$: $|3 - 9| + |3 - 1| + |3 - 3| + |3 - 7| = 6 + 2 + 0 + 4 = 12$.

To report this solution, the procedure `find_maximum` should make the following call to `allocate_tickets`:

- `allocate_tickets([[-1, 0], [0, -1], [0, -1], [-1, 0]])`

Finally, the procedure `find_maximum` should return 12.

Constraints

- $2 \leq n \leq 1500$ and n is even.
- $1 \leq k \leq m \leq 1500$
- $0 \leq x[i][j] \leq 10^9$ (for all $0 \leq i \leq n - 1$ and $0 \leq j \leq m - 1$)
- $x[i][j - 1] \leq x[i][j]$ (for all $0 \leq i \leq n - 1$ and $1 \leq j \leq m - 1$)

Subtasks

1. (11 points) $m = 1$
2. (16 points) $k = 1$
3. (14 points) $0 \leq x[i][j] \leq 1$ (for all $0 \leq i \leq n - 1$ and $0 \leq j \leq m - 1$)
4. (14 points) $k = m$
5. (12 points) $n, m \leq 80$

6. (23 points) $n, m \leq 300$
7. (10 points) No additional constraints.

Sample grader

The sample grader reads the input in the following format:

- line 1: n m k
- line $2 + i$ ($0 \leq i \leq n - 1$): $x[i][0]$ $x[i][1]$ $\ldots$ $x[i][m - 1]$

The sample grader prints your answer in the following format:

- line 1: the return value of `find_maximum`
- line $2 + i$ ($0 \leq i \leq n - 1$): $s[i][0]$ $s[i][1]$ $\ldots$ $s[i][m - 1]$

Packing Biscuits (biscuits)

Aunty Khong is organising a competition with x participants, and wants to give each participant a **bag of biscuits**. There are k different types of biscuits, numbered from 0 to $k-1$. Each biscuit of type i ($0 \le i \le k-1$) has a **tastiness value** of 2^i. Aunty Khong has $a[i]$ (possibly zero) biscuits of type i in her pantry.

Each of Aunty Khong's bags will contain zero or more biscuits of each type. The total number of biscuits of type i in all the bags must not exceed $a[i]$. The sum of tastiness values of all biscuits in a bag is called the **total tastiness** of the bag.

Help Aunty Khong find out how many different values of y exist, such that it is possible to pack x bags of biscuits, each having total tastiness equal to y.

Implementation Details

You should implement the following procedure:

```
int64 count_tastiness(int64 x, int64[] a)
```

- x: the number of bags of biscuits to pack.
- a: an array of length k. For $0 \le i \le k-1$, $a[i]$ denotes the number of biscuits of type i in the pantry.
- The procedure should return the number of different values of y, such that Aunty can pack x bags of biscuits, each one having a total tastiness of y.
- The procedure is called a total of q times (see Constraints and Subtasks sections for the allowed values of q). Each of these calls should be treated as a separate scenario.

Examples

Example 1

Consider the following call:

```
count_tastiness(3, [5, 2, 1])
```

This means that Aunty wants to pack 3 bags, and there are 3 types of biscuits in the pantry:

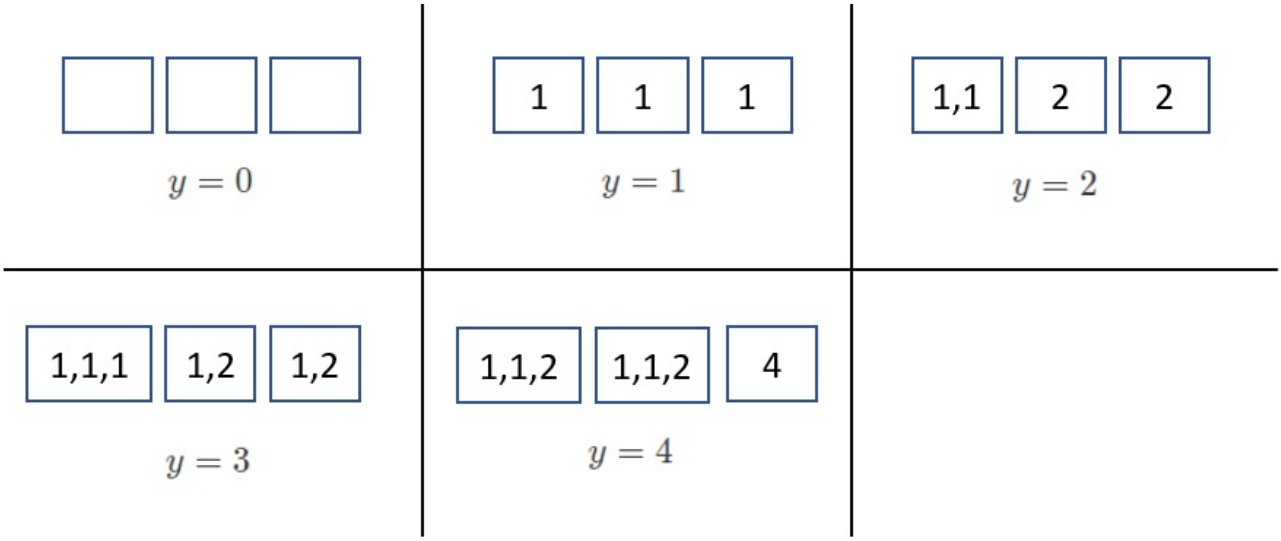
- 5 biscuits of type 0, each having a tastiness value 1,

- 2 biscuits of type 1, each having a tastiness value 2,
- 1 biscuit of type 2, having a tastiness value 4.

The possible values of y are $[0, 1, 2, 3, 4]$. For instance, in order to pack 3 bags of total tastiness 3, Aunty can pack:

- one bag containing three biscuits of type 0, and
- two bags, each containing one biscuit of type 0 and one biscuit of type 1.

Since there are 5 possible values of y, the procedure should return 5.



Example 2

Consider the following call:

```
count_tastiness(2, [2, 1, 2])
```

This means that Aunty wants to pack 2 bags, and there are 3 types of biscuits in the pantry:

- 2 biscuits of type 0, each having a tastiness value 1,
- 1 biscuit of type 1, having a tastiness value 2,
- 2 biscuits of type 2, each having a tastiness value 4.

The possible values of y are $[0, 1, 2, 4, 5, 6]$. Since there are 6 possible values of y, the procedure should return 6.

Constraints

- $1 \leq k \leq 60$
- $1 \leq q \leq 1000$
- $1 \leq x \leq 10^{18}$
- $0 \leq a[i] \leq 10^{18}$ (for all $0 \leq i \leq k - 1$)

- For each call to `count_tastiness`, the sum of tastiness values of all biscuits in the pantry does not exceed 10^{18}.

Subtasks

1. (9 points) $q \leq 10$, and for each call to `count_tastiness`, the sum of tastiness values of all biscuits in the pantry does not exceed $100\ 000$.
2. (12 points) $x = 1$, $q \leq 10$
3. (21 points) $x \leq 10\ 000$, $q \leq 10$
4. (35 points) The correct return value of each call to `count_tastiness` does not exceed $200\ 000$.
5. (23 points) No additional constraints.

Sample grader

The sample grader reads the input in the following format. The first line contains an integer q. After that, q pairs of lines follow, and each pair describes a single scenario in the following format:

- line 1: k x
- line 2: $a[0]$ $a[1]$ $\ldots$ $a[k-1]$

The output of the sample grader is in the following format:

- line i ($1 \leq i \leq q$): return value of `count_tastiness` for the i-th scenario in the input.